## Synchronouns Programming

- It is a type of programming in which code execution occurs in a single, sequential order.
- When synchronouns method is called:

Program waits for method to complete before continuing to execute the next line of code.

- This mean that the program will back, or pause, until the method completes.

**Asynchronouns programming**

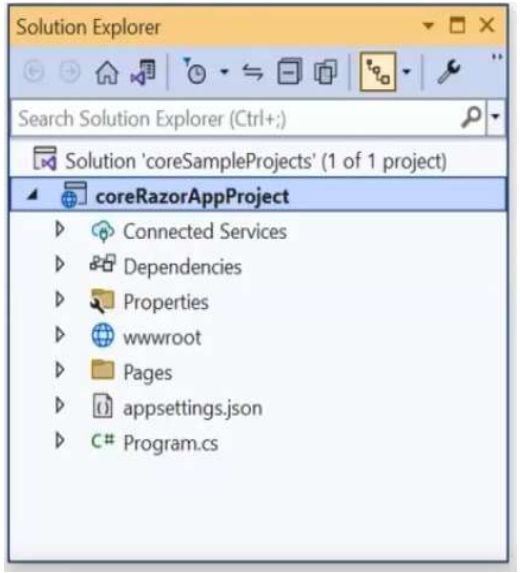
- It is an efficient approach, towards activities blocked or access is delayed.
- If an activity is blocked like this in a synchronouns process.
- Then the complete application waits and it takes more time.
- The application stops responding.
- Using the asynchronouns approach, the applications continue with other tasks as well.

# SYNCHRONOUS vs. ASYNCHRONOUS

| Synchronous Programming | Asynchronous Programming |
|---|---|
| Blocking | Non-blocking |
| Single threaded | Can be multi-threaded |
| Simple to understand | Complex |
| May result in UI freezes | Can improve UI responsiveness |
| Limited to the capabilities of a single CPU | Can take advantage of multiple CPU cores |
| Exceptions are raised immediately | Exceptions can be wrapped in a task and raised later |
| Does not use tasks | Uses tasks to represent async unit of work |

# ASP.NET Core Project Folder Structure

- The details of the default project can be seen in the solution explorer.
- It displays all the projects related to a single solution.
- Project Folder structure is organized in a hierarchical manner.
- Each level representing a different aspect of the application.

# Additional information

## ASP.NET Core Web App (Model-View-Controller)

C# Linux macOS Windows Cloud Service Web

Framework

.NET 6.0 (Long Term Support)

Authentication type

None

- [ ] Configure for HTTPS
- [ ] Enable Docker

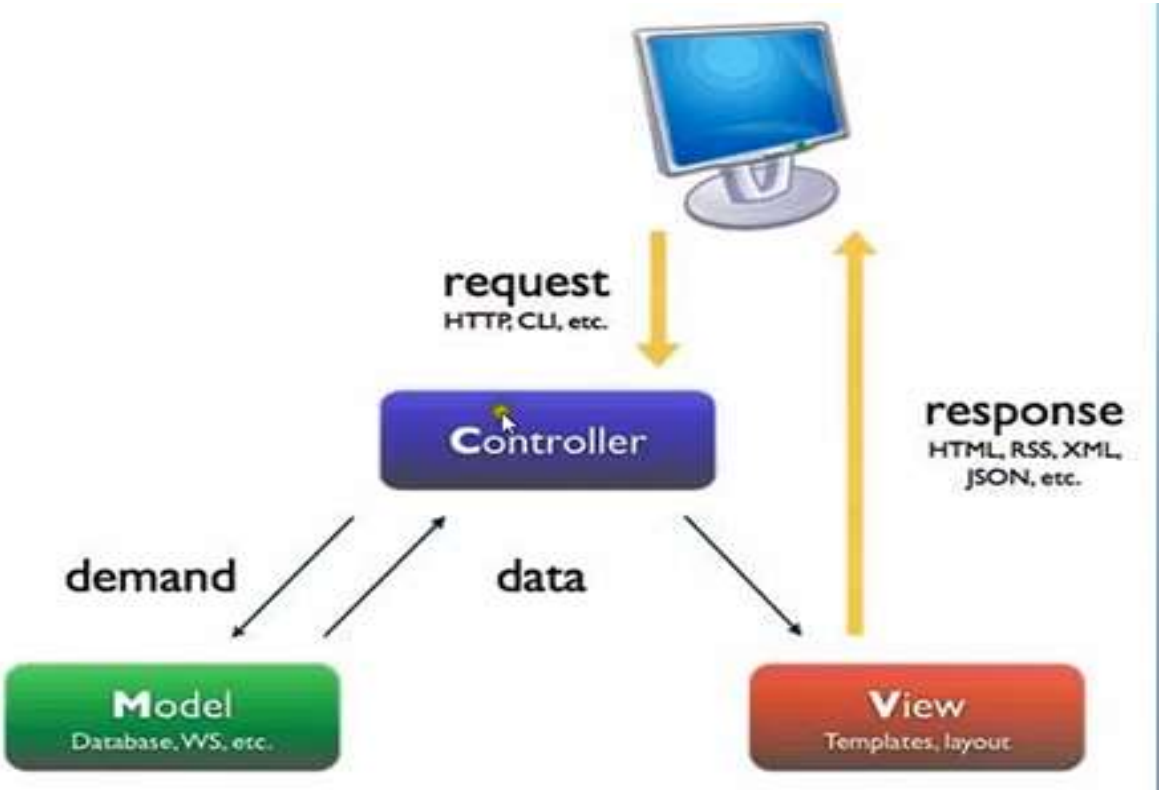
Docker OS

Linux

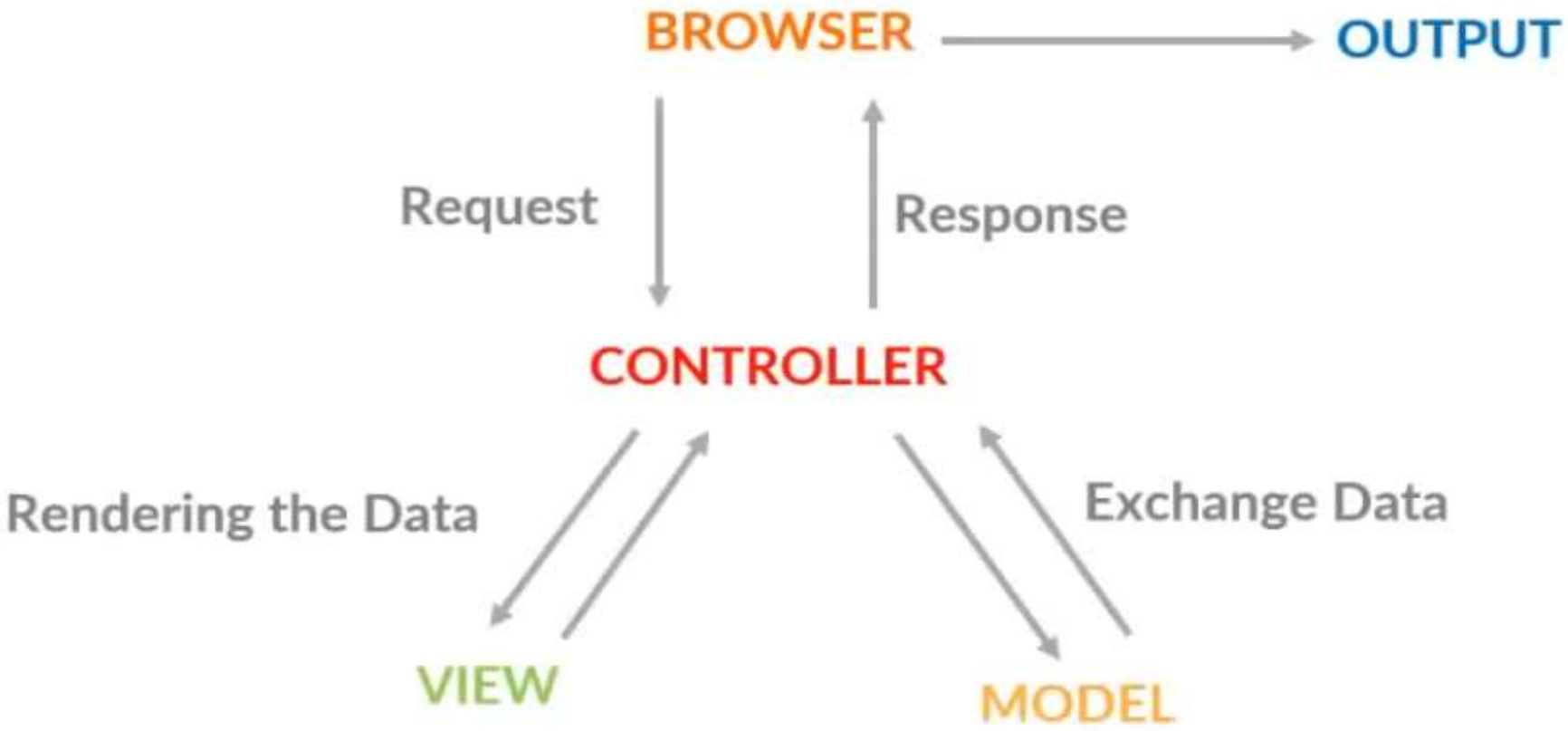
- [ ] Do not use top-level statements

# ASP.NET MVC LIFE CYCLE

Look MVC is totally action based, our request comes to controller and then action executes if it needs data then it can retrieve the data from model and then show on the views.



# Model - View – Controller Communication



```
public string str()
        {
            return "fun to learning MVC.Net";
        }



public int Add()
        {
            int a = 8, b = 10;
            return a + b;
        }
```

# Ex

Index.cshtml* | HomeController.cs

```
4       }
5
6       <!DOCTYPE html>
7
8       <html>
9       <head>
10          <meta name="viewport" content="width=device-width" />
11          <title>Index</title>
12      </head>
13      <body>
14          <div>
15              <h1>Wellcome to series MVC5</h1>
16          </div>
17      </body>
18      </html>
```

```
public ActionResult Index()
        {
            return View();
        }

public RedirectResult ShowInternal()
        {
            return
Redirect("/Home/Index");
        }
public RedirectResult ShowExternal()
        {
            return
Redirect("https://www.google.com/");
        }
```

Ex:  TestController:

```
public ViewResult List()
{
    return View();
}
```

HomeController:

```
public ContentResult Details()
{
    int x = Convert.ToInt32(HttpContext.Request.Query["id"]);
    string country = "";
    switch (x)
    {
        case 1: country = "Germany"; break;
        case 2: country = "Italy"; break;
        case 3: country = "France"; break;
        case 4: country = "Syria"; break;
        default: country = "Sorry, you do not belong to any country."; break;
    }
    return Content(country);
}
```

View for List:

```
<table border="1">
    <tr>
        <th>Name</th>
        <th>Details</th>
    </tr>
    <tr>
        <td>Osama</td>
        <td><a href="/Home/Details?id=1">Details</a></td>
    </tr>
    <tr>
        <td>Sami</td>
        <td><a href="/Home/Details?id=2">Details</a></td>
    </tr>
    <tr>
        <td>Samer</td>
        <td><a href="/Home/Details?id=3">Details</a></td>
    </tr>
    <tr>
        <td>Abeer</td>
        <td><a href="/Home/Details?id=4">Details</a></td>
    </tr>
</table>
```

# Model

## Model represents the data

- Model represents the shape of the data.
- A Model is a class that represents the data and business logic of the application.
- Responsible for defining the data entities, validating input, and performing database operations.
- Used in conjunction with data access technologies such as Entity Framework, LINQ or SQ.

## What are Models?

- A model is represented by a C# class that contains properties to store data needed for an operation.
- Strongly typed views typically use ViewModel types designed to contain the data to display on that view.
- The controller creates and populates these ViewModel objects from the model
- ASP.NET Core MVC uses model binding to convert request data (form values, query string parameters, etc.) into objects that the controller can handel.

# View

## View represents the User Interface

- View is responsible for rendering the user interface of the application.
- View display model data to the user and also enables them to modify them.
- A view is typically defined as an HTML template that includes server-side code to render data.
- Views can use a variety of technologies to render HTML. Including Razor, ASPX, and others.
- Client-side technologies such as JavaScript and CSS can be used to enhance the user interface.

**What are Views?**

- Views are responsible for presenting content through the user interface.
- Found in a folder called Views and a .cshtml extension.
- They use the Razor view engine to embed .NET code in HTML markup.
  - A hybrid HTML file that allows C# code to be written as needed.
- Designed to display data (static or dynamic) with a little logic as possible.

**What are the controllers?**

- Controllers handle requests, work with the model, and ultimately select a view to render.

- Views only display information, so the controller:
  - Handles a user request(eg.www.website.com/about).
  - Invokes a method called an **action**
  - Performs business logic as needed,
  - Collates data.
  - Then returns a view that corresponds with the web address, along with relevant data.

# Controller

## Controller represents the Request handler

- A controller is responsible for handling user input and updating the model and the View.
- It is the Component that handles requests from the client.
- Performs necessary operations on the model, and selects the right view to render the response.
- Controller is defined as a class that inherits from the base controller class in ASP.NET MVC.
- Includes action methods that correspond to user requests and perform operations on the model.

# Model – View – Controller Communication

- **Model to Model**:
  - We can communication form model to model via parameters/composition.
- **Model to View:**
  - To communicate Model to View, you have to follow the path**:**

    **Model > Controller > View**.
  - We cant directly move from Model to View.
  - First, the Model object is made in the controller and then it is passed to View.
  - **We can pass the data or communicate from Model to View by these three steps.**
    - **Take the object in the action of a Controller.**
    - **Pass Model object as a parameter to View.**
    - **Use @model to include Model on the View Page.**

# Model – View – Controller Communication

- **Model to Controller:**
  - Create an object of Model class in Controller to access the Model in Controller.
- **View to Model**:
  - To communicate View to Model, you have to follow the path:

    View > Controller > Model.
  - We can't directly move from View to Model.
  - First, you have to submit data to the controller and then pass it to the model.
  - **To pass the data from View to Model, you have to follow these three steps:**
    - **Submit Html form to controller.**
    - **Create an Object of Model in Controller.**
    - **Pass Values to the Model object.**

# Model – View – Controller Communication

- **View to View**
  - We can move from one view to another view by using partial views.
- **View to Controller**:
  - We can move data from view to controller by submitting forms to controllers or by:
  - JSON
  - AJAX Calls
  - JavaScript
  - Partial Views

## Best Practices

- Follow the naming conventions:
  - Controller names must follow the format RoutNameController.
  - View folder must exist with a name of RouteName
  - Each action defined in RouteNameController should have matching view.
    - Eg. An Index action must have a matching Index.cshtml view file.
  - Create folders for files of similar natures.

## Model – View – Controller Communication

- Controller to Model:

  we can move from Controller to Model just like we move from Model to Controller.

. View to Controller:

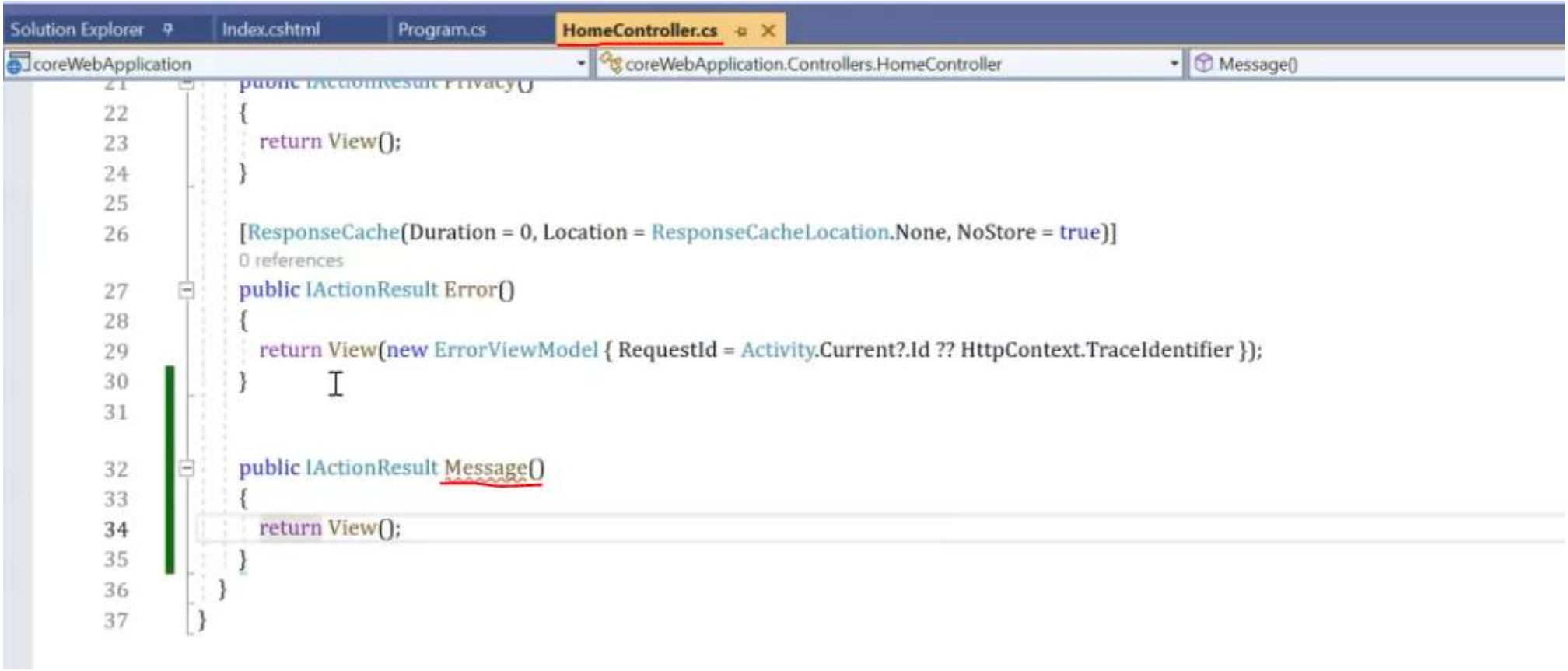
  we can move data from controller to view in the following ways:

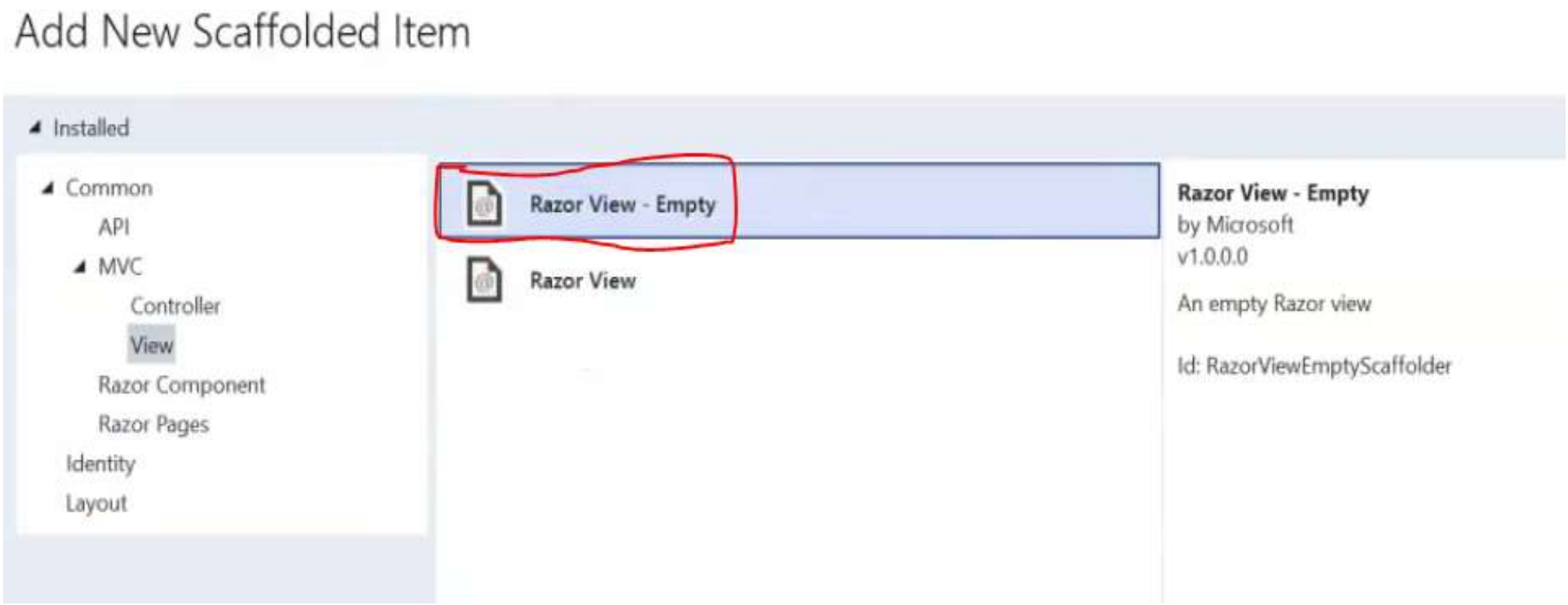
  - By using viewBag
  - ViewData
  - TempData

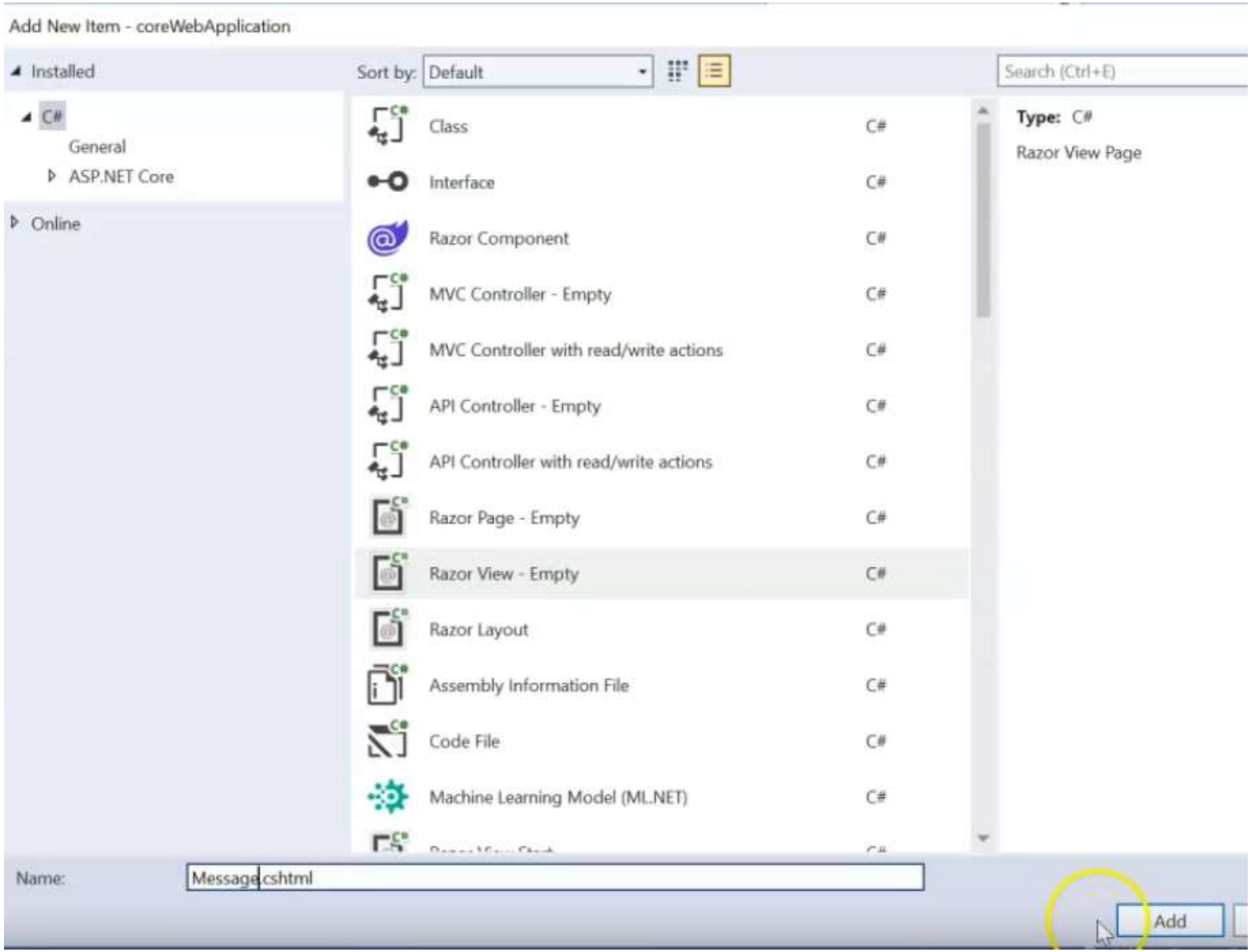
. Controller to Controller:

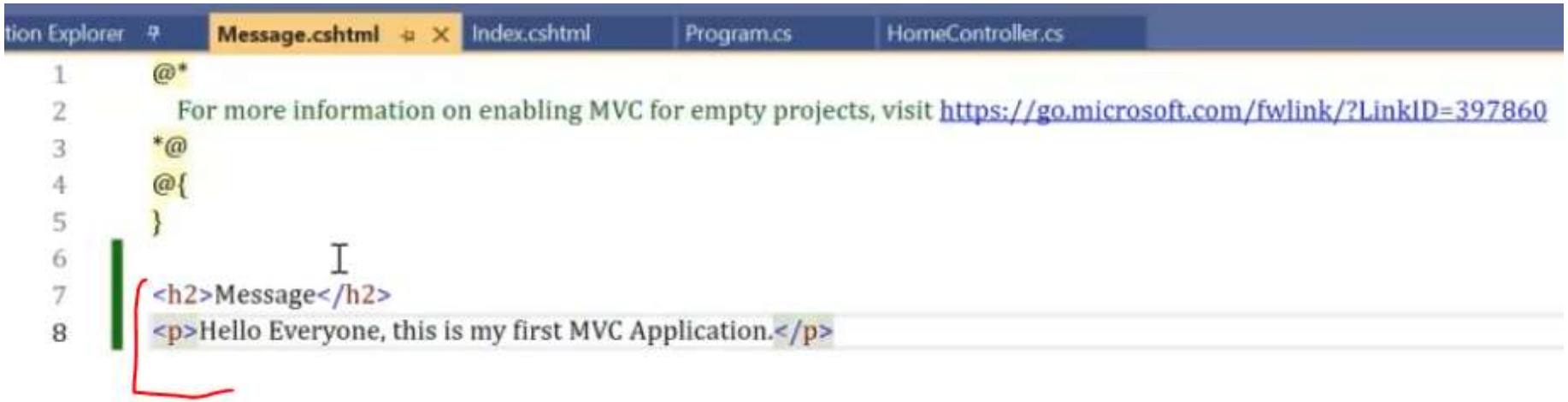
  we can move one controller to another by using RedirectToAction():

  and then pass the name of the specific action.


Solution Explorer
Index.cshtml
Program.cs
HomeController.cs
coreWebApplication
coreWebApplication.Controllers.HomeController
Message()
public IActionResult Privacy()
{
return View();
}
[ResponseCache(Duration = 0, Location = ResponseCacheLocation.None, NoStore = true)]
0 references
public IActionResult Error()
{
return View(new ErrorViewModel { RequestId = Activity.Current?.Id ?? HttpContext.TraceIdentifier });
}
public IActionResult Message()
{
return View();
}
}
}


Add New Scaffolded Item
Installed
Common
API
MVC
Controller
View
Razor Component
Razor Pages
Identity
Layout
Razor View - Empty
Razor View
Razor View - Empty
by Microsoft
v1.0.0.0
An empty Razor view
Id: RazorViewEmptyScaffolder


Add New Item - coreWebApplication
Installed
C#
General
ASP.NET Core
Online
Sort by: Default
Search (Ctrl+E)
Class C#
Interface C#
Razor Component C#
MVC Controller - Empty C#
MVC Controller with read/write actions C#
API Controller - Empty C#
API Controller with read/write actions C#
Razor Page - Empty C#
Razor View - Empty C#
Razor Layout C#
Assembly Information File C#
Code File C#
Machine Learning Model (ML.NET) C#
Type: C#
Razor View Page
Name: Message.cshtml
Add


tion Explorer
Message.cshtml
Index.cshtml
Program.cs
HomeController.cs
@*
For more information on enabling MVC for empty projects, visit https://go.microsoft.com/fwlink/?LinkID=397860
*@
@{
}
<h2>Message</h2>
Hello Everyone, this is my first MVC Application.


coreWebApplication Home Privacy

# Message

Hello Everyone, this is my first MVC Application.

**Razor?**

- Razor pages- this project type implements the Model-View-ViewModel pattern where C# is written in a class file, directly attached to the frontend file.
- MVC – project type based on the Model-View-Controller pattern.
- API-project type used to create APIs (primarily RESTful)

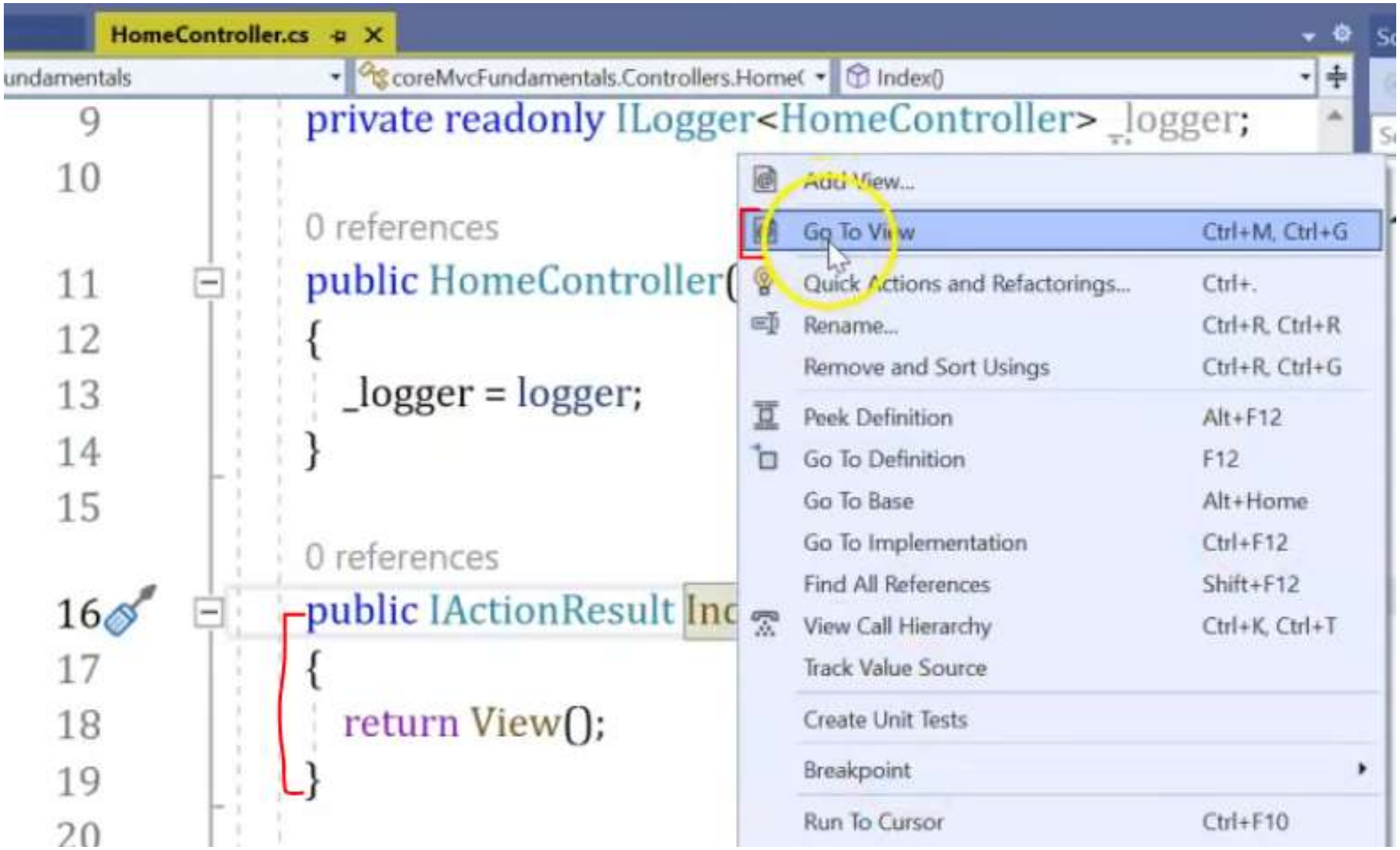
## Action Methods

- an action is a method that responds to user action/activity.
- My method in a controller which has public access modifier acts as an action method.
- **They are like any other normal methods with the following restrictions:**
  - **Action method must be public. It cannot be private or protected.**
  - **Action method cannot be overloaded.**
  - **Action method cannot be a static method.**

## Default Action Methods

- every controller can have default action method as per configured route in the RouteConfig.
- By default, the index() method is a default action method for any controller.
- Also, you can change the default action name as per your requirement in the RoutConfig class.

```
0 references
public IActionResult Index()
{
    return View();
}
```

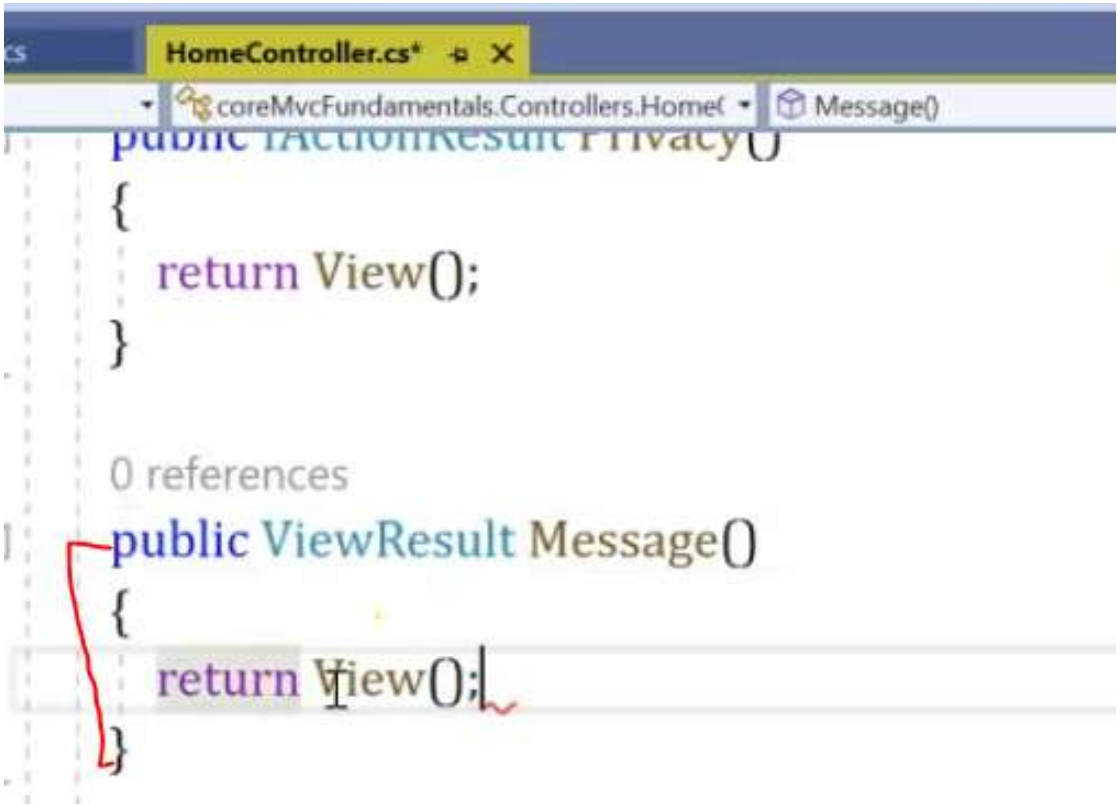

HomeController.cs
undamentals
coreMvcFundamentals.Controllers.Home(
Index()
9 private readonly ILogger<HomeController> _logger;
10
0 references
11 public HomeController(
12 {
13 _logger = logger;
14 }
15
0 references
16 public IActionResult Inc
17 {
18 return View();
19 }
20
Add View...
Go To View Ctrl+M, Ctrl+G
Quick Actions and Refactorings... Ctrl+.
Rename... Ctrl+R, Ctrl+R
Remove and Sort Usings Ctrl+R, Ctrl+G
Peek Definition Alt+F12
Go To Definition F12
Go To Base Alt+Home
Go To Implementation Ctrl+F12
Find All References Shift+F12
View Call Hierarchy Ctrl+K, Ctrl+T
Track Value Source
Create Unit Tests
Breakpoint
Run To Cursor Ctrl+F10

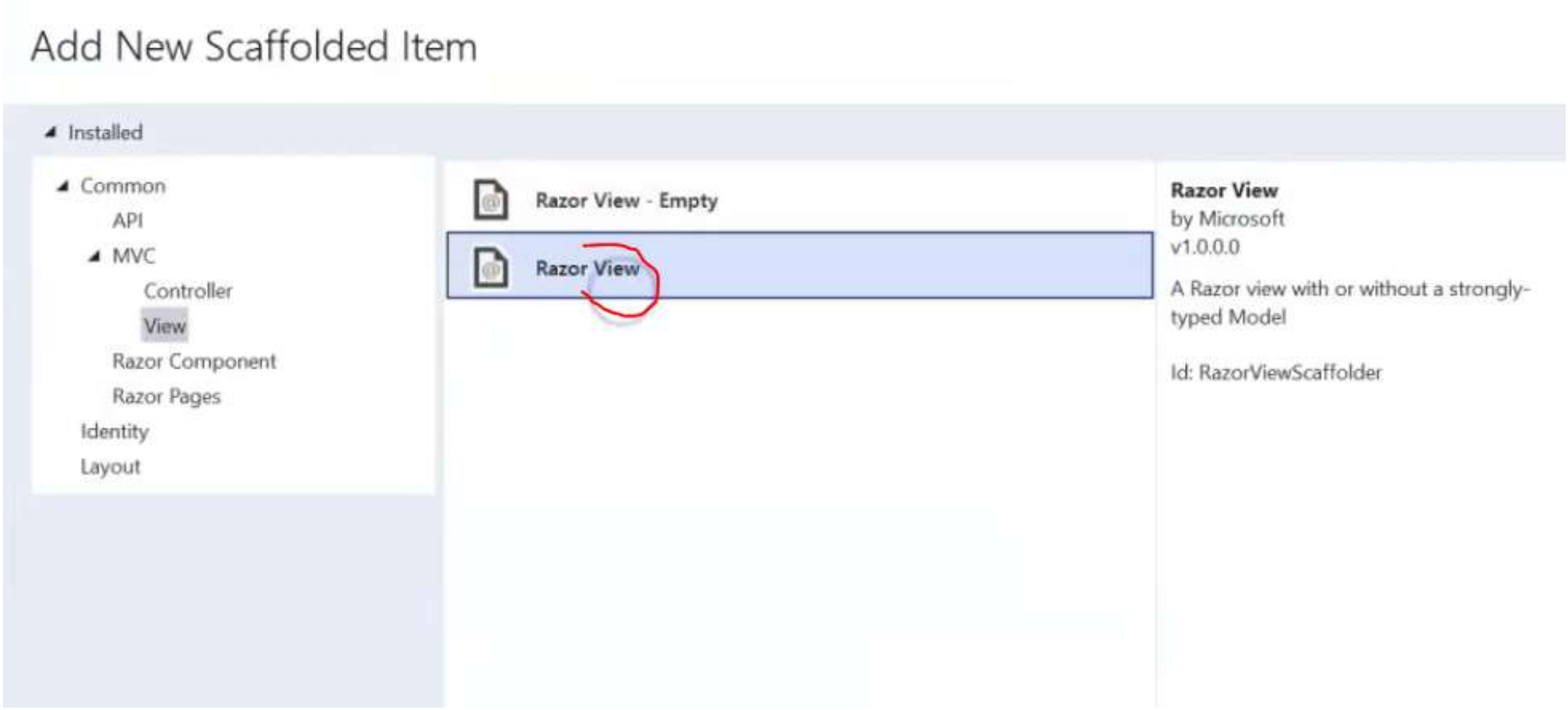

Index.cshtml
Program.cs
HomeController.cs
1 @{
2 ViewData["Title"] = "Home Page";
3 }
4
5 
6 <h1 class="display-4">Welcome</h1>
7 <p>Learn about <a href="https://docs.microsoft.com/aspnet
8 
9

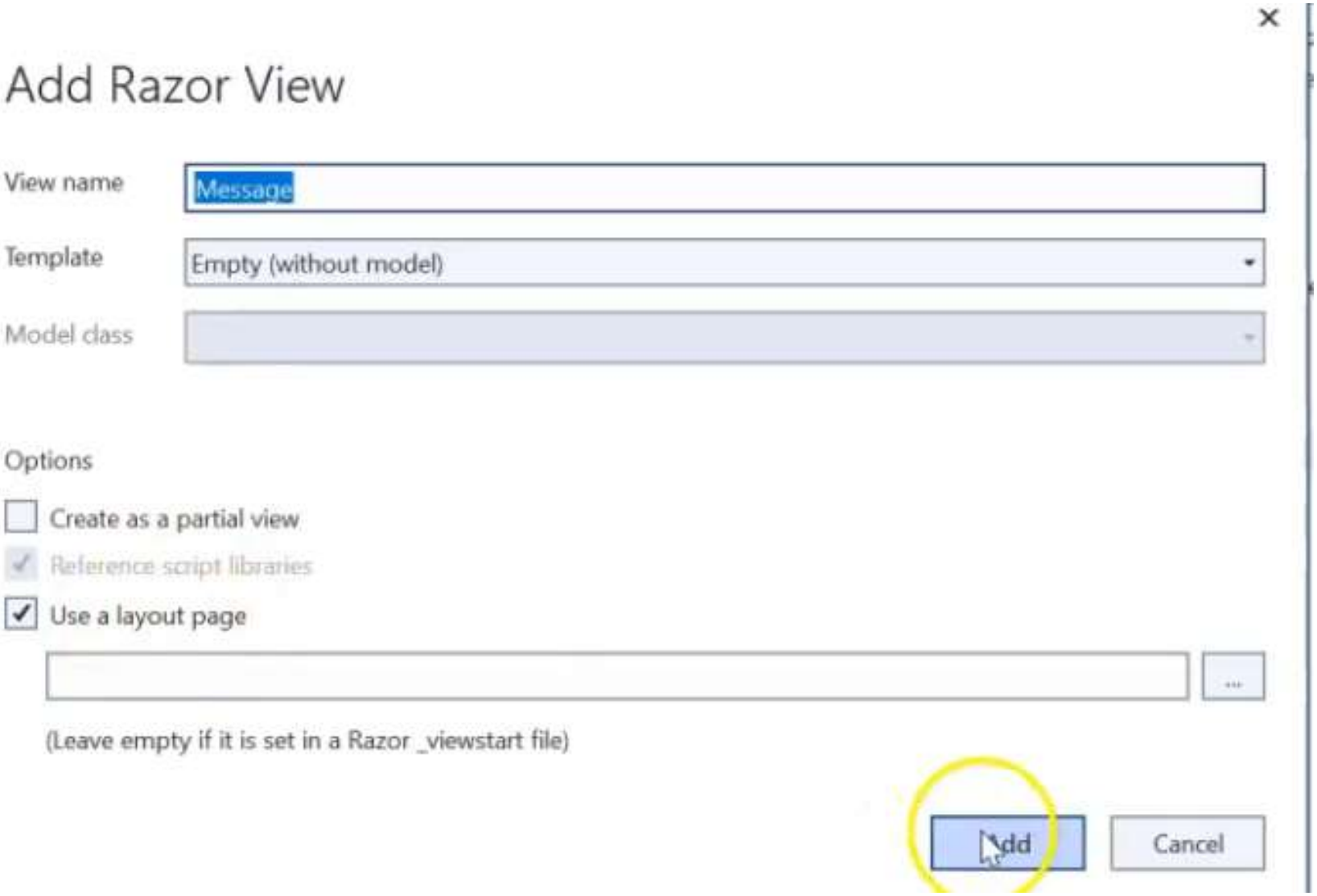

HomeController.cs
coreMvcFundamentals.Controllers.Home
Message()
public IActionResult Index()
{
return View();
}
0 references
public IActionResult Privacy()
{
return View();
}
0 references
public string Message()
{
return "Hello World";
}


localhost:5270/Home/Message
Apps
Bhawna Gunwani O...
Shailendra
Microso

Hello World


HomeController.cs*
coreMvcFundamentals.Controllers.Home(
Message()
{
return View();
}
0 references
public ViewResult Message()
{
return View();
}

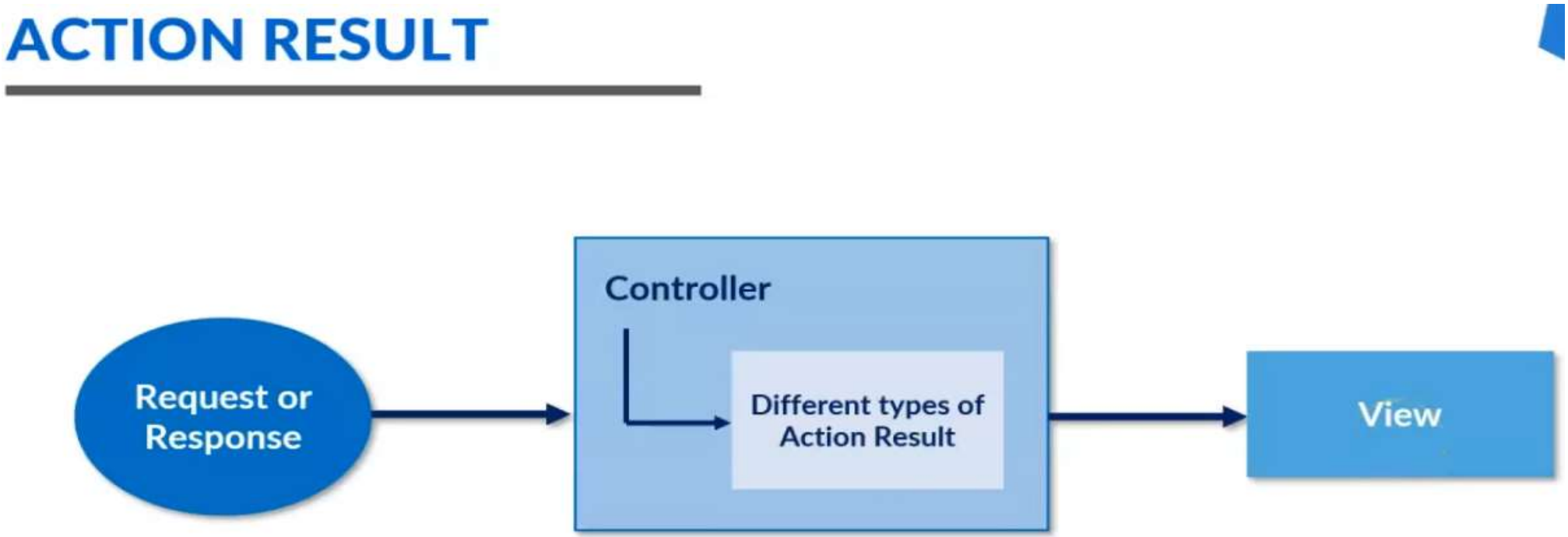

Add New Scaffolded Item
Installed
Common
API
MVC
Controller
View
Razor Component
Razor Pages
Identity
Layout
Razor View - Empty
Razor View
Razor View
by Microsoft
v1.0.0.0
A Razor view with or without a strongly-typed Model
Id: RazorViewScaffolder


Add Razor View
View name
Message
Template
Empty (without model)
Model class
Options
Create as a partial view
Reference script libraries
Use a layout page
...
(Leave empty if it is set in a Razor _viewstart file)
Add
Cancel

# Action Result

- MVC Framework includes various Result classes, which can be returned from an action method.
- Result Classes represents different types of responses, such as HTML, file, string, JSON etc.
- ActionResult class is a base class of all the above result classes,
- So it can be the return type of action method that returns any result.
- However, you can specify the appropriate result class as a return type of action method.
- The base controller class includes the view() method along with other methods that return a particular types of result.

# Action Result



# Action Result

The following table lists all the result classes with their base controller methods:

| Result Class | Description | Base Controller Method |
|---|---|---|
| **ViewResult** | Represents HTML and markup. | View() |
| **EmptyResult** | Represents No response. | |
| **ContentResult** | Result string literal. | Content() |
| **FileContentResult** | Represents the content of a file. | File() |
| **RedirectResult** | Represents a redirection to a new URL. | Redirect() |
| **JsonResult** | Represent JSON that can be used in AJAX. | Json() |
| **PartialViewResult** | Returns HTML from Partial view. | PartialView() |
| **JavaScriptResult** | Represents a JavaScript script. | JavaScript() |
| **HttpUnauthorizedResult** | Represents HTTP 403 response. | |